

Section 14.6: Infrastructure & Platform Systems

Part 6 of 8 in the System Design Interview Series

Executive Overview

Infrastructure systems power distributed applications and must be correct above all — bugs cascade to thousands of dependent services. This section covers message queues, job schedulers, distributed locking, and observability systems. For consensus-based systems (locking, leader election), give extra weight to the correctness arguments: why quorums work, why fencing tokens prevent split-brain, and what “linearisability” actually means in practice.

1. Design a Distributed Message Queue (Kafka-Style)

Requirements Analysis

Dimension	Requirement
Functional	Publish/consume messages; preserve per-key order
Scale	Trillions of messages/day
Durability	Persistent storage; configurable replication
Delivery Semantics	At-least-once by default; exactly-once available

Core Architecture

```
Producers → [Broker 1 (Leader)] → Consumer Group A
              ↓ replicate
            [Broker 2 (Follower)]
            [Broker 3 (Follower)]
```

Topic = logical stream. Partition = physical shard of that stream.

A topic with 100 partitions allows 100 consumers in a group to read in parallel. Partitions are the unit of parallelism, durability, and ordering.

Partitioning and Ordering

Key-based partitioning:

```
partition = hash(message_key) % num_partitions
```

Example:

```
Key = user_id = "12345"
```

```
hash("12345") % 100 = partition 73
```

Guarantee: All messages for user_id "12345" go to partition 73

Result: Strictly ordered per user_id across producers

Why ordering matters: If user sends messages [A, B, C] and they land on different partitions, a consumer might process them as C, A, B — breaking causality. Key-based partitioning solves this.

Trade-off: Hot keys (one very active user) create hot partitions. Mitigation: Salted keys (`user_id + random_suffix`), but this breaks ordering. Choose based on requirement.

Replication (ISR Model) — Correctness Argument

ISR = In-Sync Replicas: the set of replicas currently caught up with the leader.

```
Producer sends message to leader (Broker 1):
```

```
Leader appends to local log
```

```
Leader sends to all followers
```

```
Followers append and send ACK
```

```
acks=all: Producer waits until ALL ISR members have written
```

```
→ Message survives any single broker failure
```

```
→ Latency: ~10ms extra for cross-broker replication
```

```
acks=1: Producer waits only for leader ACK
```

```
→ Message lost if leader fails before replication completes
```

```
→ Lower latency (~1ms)
```

Proof of correctness for acks=all: If the leader fails after replication to all ISR members, any ISR member can become the new leader with the full message history. No message loss is possible. If a follower falls behind (network partition), it's removed from ISR. The leader continues with the remaining ISR members. The lagging follower is not promoted until it fully catches up.

Minimum ISR setting: `min.insync.replicas=2` means the producer gets an error if fewer than 2 replicas are in sync. Prevents silent under-replication. Set to $\lceil N/2 \rceil + 1$ for quorum-based durability.

Consumer Groups

Topic: user-events (100 partitions)

Consumer Group "analytics-pipeline" (3 consumers):

Consumer 1 → partitions 0-32

Consumer 2 → partitions 33-66

Consumer 3 → partitions 67-99

Each partition is consumed by exactly one consumer in the group

→ Parallel processing; no duplicate consumption within group

→ Multiple groups can consume same topic independently (each has own offset)

Offset management: Each consumer commits its read position (`offset`) per partition. On restart, it resumes from last committed offset. Commit too rarely → reprocess messages on failure. Commit too eagerly → may miss messages in a crash between read and commit.

Delivery Semantics Comparison

Semantic	Guarantee	How	Trade-off
At-most-once	No duplicates; may lose messages	Don't retry on failure	Data loss possible
At-least-once	No loss; may duplicate	Retry until ACK	Consumer must be idempotent
Exactly-once	No loss, no duplicates	Transactional producer + EOS consumer	Higher latency, complex setup

Exactly-once implementation: Producer uses idempotent mode (deduplicates by producer epoch + sequence). Consumer uses Kafka transactions: consume → process → produce → commit offset atomically. Any failure rolls back all three steps.

Interview Problem 1 — Kafka for Real-Time Analytics

“Your team wants to build a real-time dashboard showing events-per-second metrics aggregated across 50 microservices. All services publish to Kafka. How do you design the aggregation pipeline?”

Solution framework:

1. **Topic design:** Each service writes to its own topic (`service-A-events` , `service-B-events`). A single `all-events` topic receives fan-in via a Kafka Streams job.

2. **Stream processing**: Kafka Streams (or Flink) consumes `all-events`, windows events into 5-second tumbling windows, counts per service, and writes aggregated results to `metrics-aggregated` topic.
3. **Dashboard read path**: Metrics consumer reads from `metrics-aggregated`, writes to Redis time-series (`TS.ADD metrics:service-A now count`). Dashboard queries Redis every 5 seconds.
4. **Backpressure**: If aggregation consumer falls behind, Kafka buffers messages. Consumer auto-scales horizontally (one consumer per partition).
5. **Late arrivals**: Use event-time watermarks. Accept events up to 30 seconds late before closing a window. Results marked as “preliminary” until window is fully closed.

2. Design a Distributed Job Scheduler

Requirements Analysis

Dimension	Requirement
Functional	Cron jobs, one-time delayed tasks, priority queues
Scale	Millions of scheduled jobs across fleet
Reliability	No job lost; exactly-once execution

Scheduling Strategies Comparison

Strategy	Accuracy	Throughput	Failure Handling	Complexity
Single scheduler	High	Poor (single node)	SPOF	Simple
Leader election (etcd/ZK)	High	Good	Automatic failover	Medium
Distributed consensus	Perfect	Poor	Expensive coordination	High

Production standard: Leader election

One scheduler node is elected leader (via etcd lease). It is the sole decision-maker for scheduling. All other nodes are hot standbys, ready to acquire the lease if the leader fails. Leader failure detected within 10–30 seconds (lease TTL).

Exactly-Once Execution

The critical pattern preventing both lost and duplicate executions:

```
-- Atomic claim: only one worker can succeed
UPDATE jobs
SET status = 'RUNNING', worker_id = 'w-123', started_at = NOW()
WHERE job_id = X AND status = 'PENDING';

-- affected_rows = 1 → this worker owns the job
-- affected_rows = 0 → another worker claimed it first → skip
```

After execution:

```
UPDATE jobs SET status = 'COMPLETE', completed_at = NOW()
WHERE job_id = X AND worker_id = 'w-123';
-- worker_id check prevents a stale worker from marking a re-claimed job as
complete
```

Zombie worker problem: A worker picks up a job, goes silent (GC pause, network partition), job times out, another worker picks it up, first worker revives and tries to complete it. The `worker_id` check prevents the zombie from overwriting the new worker's result.

Priority Queue Implementation

```
# Each priority level as a separate sorted set
ZADD jobs:critical timestamp job_id_1
ZADD jobs:normal    timestamp job_id_2

# Scheduler polls in priority order
ZRANGEBYSCORE jobs:critical -inf now LIMIT 0 100
ZRANGEBYSCORE jobs:normal   -inf now LIMIT 0 100
```

Interview Problem 2 — Cron at Scale

“Airbnb uses a distributed cron system to run millions of scheduled jobs (like ‘send check-in reminder 24 hours before arrival’). How do you design this?”

Solution framework:

1. **Job storage:** Jobs stored in a DB with `next_run_at` index. Scheduler polls `SELECT * WHERE next_run_at <= NOW() AND status = 'PENDING' LIMIT 1000`.

2. **Tick generation:** A leader-elected tick generator fires every 1 second, queries due jobs, and pushes to worker queues.
3. **Worker pool:** Workers pull from queues, atomically claim jobs (UPDATE as above), execute, mark complete, schedule next run.
4. **Next run calculation:** After completion, compute `next_run_at = cron_next(expression, completed_at)` and insert a new PENDING job record (or update the existing one with new `next_run_at`).
5. **Missed jobs recovery:** On startup, scan for jobs where `next_run_at < NOW() - 5min AND status = 'PENDING'`. These are missed jobs. Run them immediately with a `MISSED` flag, then schedule their next occurrence normally.

3. Design a Distributed Locking Service (ZooKeeper / etcd)

Requirements Analysis

Dimension	Requirement
Functional	Distributed locks, leader election, config watch
Consistency	Linearisable — locks must be exactly exclusive
Availability	Survive $N/2$ node failures (quorum)

What “Linearisable” Means (With a Proof Sketch)

Linearisability: Every operation appears to take effect atomically at some single instant in real time, between its invocation and completion. To a distributed system client, it looks like there’s only one copy of the data.

Why this matters for locks: If `acquire(lock)` returns “success” to Client A, no other client can see the lock as “free” until A releases it — even if A’s ACK was delayed in transit.

ZAB protocol (ZooKeeper’s consensus): 1. All writes go through a single elected leader 2. Leader proposes a change to all followers 3. At least $\lfloor N/2 \rfloor + 1$ followers must ACK before the leader commits 4. Leader broadcasts the commit; followers apply it

Correctness argument: Any two quorums of size $\lfloor N/2 \rfloor + 1$ must overlap by at least 1 node. That overlapping node has seen the latest committed write. Therefore, any read quorum will include a node with the latest value. No stale reads are possible in a linearisable read.

Numeric example (N=5): - Quorum size = 3 - Write quorum $W_1 = \{A, B, C\}$ - Read quorum $W_2 = \{C, D, E\}$ - Overlap: $\{C\}$ — node C in both quorums - C has the latest committed write → read returns correct value ✓

Lock Acquisition via Ephemeral Sequential Nodes

```
Client wants lock on "resource-42":
1. Create /locks/resource-42/lock-0000000001 (ephemeral sequential)
2. List all children of /locks/resource-42/
3. If my node has the lowest sequence number → I hold the lock
4. Else: Watch the node with the next-lower sequence number
5. When that node is deleted → re-check and acquire if now lowest

Release:
  Delete my ephemeral node
  → ZooKeeper notifies next waiter → they check and acquire
```

Why ephemeral nodes? If the client dies, its session expires, its ephemeral nodes are deleted, and the lock is automatically released. No manual cleanup. No permanent locks from dead clients.

Fencing Tokens — Preventing Stale Lock Owners

The problem: Client A holds a lock. A GC pause causes A to miss its heartbeat. Lock expires. Client B acquires lock and starts writing. A resumes from GC, still thinks it holds the lock, and also writes. Data corruption.

```
Lock service issues fencing tokens (monotonically increasing integers):
  Client A acquires lock → receives token 42
  Lock expires → Client B acquires lock → receives token 43

Data store enforces:
  Client A writes with token 42 → REJECTED (last seen token was 43)
  Client B writes with token 43 → ACCEPTED

Proof of correctness:
  Tokens are monotonically increasing
  Data store only accepts writes with token ≥ last_seen_token
  Any write from a stale holder carries a lower token → automatically rejected
```

Implementation:

```
def write_with_fence(resource_id, data, fence_token):
    # Lua script for atomicity
    result = redis.eval("""
        local current_fence = tonumber(redis.call('GET', 'fence:' .. KEYS[1]) or
0)
        if tonumber(ARGV[2]) < current_fence then
```

```

        return 0  -- Stale token, reject
    end
    redis.call('SET', 'fence:' .. KEYS[1], ARGV[2])
    redis.call('SET', 'data:' .. KEYS[1], ARGV[1])
    return 1
"", 1, resource_id, data, fence_token)
return result == 1

```

Interview Problem 3 — Redis Redlock vs etcd for Distributed Locking

“Your team is debating between Redis Redlock and etcd for distributed locking in a microservices system. How do you choose?”

Solution framework:

Criterion	Redis Redlock	etcd
Consistency	Approximate (clock-dependent)	Linearisable
Latency	~1ms	~5–20ms
Failure modes	Clock drift can violate safety	Leader election adds latency
Operational complexity	Moderate (5-node Redis cluster)	Higher (etcd cluster management)
Use when	Microservices, non-critical locks	Kubernetes, critical coordination

When Redlock is risky: GC pauses longer than the lock TTL can cause two clients to simultaneously hold the lock even with Redlock. Martin Kleppmann’s analysis showed this is possible. Mitigation: use fencing tokens at the data store level regardless of which lock service you use.

Recommendation: Use etcd or ZooKeeper when correctness is non-negotiable (financial systems, cluster coordination). Use Redis locks with fencing tokens for performance-sensitive but correctness-best-effort scenarios (rate limiting, idempotency dedup).

Section Summary: Infrastructure Platform Patterns

Problem	Pattern	Key Correctness Concern
Message ordering	Partition by key	Hot key hot partition skew

Problem	Pattern	Key Correctness Concern
Exactly-once delivery	Transactional producer + EOS	Idempotent consumer still required
Distributed job scheduling	Leader election + atomic claim	Zombie workers, missed job recovery
Distributed locking	Ephemeral sequential nodes	Fencing tokens prevent stale writes
Lock correctness	Linearisable quorum (etcd)	Clock drift invalidates Redlock in edge cases